Advanced algorithms **Master DSC/MLDM**

Project - 2026-2027

---

1 Description

The objective of this project is to implement some algorithms for solving the *Minimum Set Cover Problem* and to provide an experimental study of their running time and quality.
`https://en.wikipedia.org/wiki/Set_cover_problem`

2 Work to do

You must program different algorithms for the Minimum Set Cover problem, including:

- The brute-force approach exploring all the possible solutions in a systematic way.
- A Branch-and-Bound version, to be designed on your own (you can actually work on multiple versions)
- A greedy algorithm that iteratively selects the subset containing the largest number of currently uncovered elements. Additional greedy variants may be implemented and compared.
- An exact dynamic programming approach, possibly including a top-down implementation with memoization.
- Another method of your choice, it could be another branch and bound solution but different from the first one.
- An additional heuristic or metaheuristic, such as randomized greedy search, local search, a genetic algorithm, or ant colony optimization.

All algorithms must output a solution, i.e. the selected subsets and the cardinality of the resulting cover.

The running time of the algorithms and the quality of the solutions given (in terms of optimality) have to be evaluated on problems of different sizes and structure (*i.e.* with different number of items, different universe sizes, numbers of subsets, subset sizes, instance densities, ...).

For this purpose, you can implement a random generator that can automatically create some instances of the problem. It should take into account the number of elements in the universe, the number of candidate subsets, and the probability that an element belongs to a particular subset (i.e. control the density).
Your evaluation must also include existing benchmark instances. For this purpose, you may use the repository `https://github.com/pinam45/USCP_instances`. This repository contains instances of both weighted and unweighted Set Cover. For this project, you must select only unweighted instances, for which all candidate subsets have unit cost.

The repository provides instances with different sizes and structural characteristics. Table 1 summarizes some of the available benchmark families. When evaluating solution quality, clearly distinguish between instances for which the optimal solution is known and those for which only a best-known solution is available. Please consult the repository documentation for the instance formats and the associated references.

Table 1: Characteristics of the benchmark instance families.

Family	Instances	Number of elements	Number of subsets	Density
E	E.1–E.5	50	500	20%
CLR	CLR10–CLR13	511–4,095	210–715	12%
CYC	CYC6–CYC11	240–28,160	192–11,264	0.04–2.1%

You have to provide a full and rigorous experimental study to compare the different algorithms and illustrate the strong and weak points of each them. In particular, it is expected that the efficiency and the quality of the solutions are studied with care. In the report, it is expected that all the algorithms used are presented clearly with their specificities/properties, notably those not seen in class, and that the experimental setup is precisely indicated and accompanied with a thorough analysis of the results.

Note also that you will have to present your project during a defense that lasts **15 minutes, no more**. The main objective is then not to re-explain the algorithms already seen in class, but rather to present and explain your results. There will be a long period of time dedicated to questions and exchanges with the group.

A graphical user interface could be proposed to illustrate the behavior of the algorithm(s), but **this is not required** and must be done after the implementation of the algorithms.

The project can be done in a group from 4 to 5 students, and each student must program **at least** one algorithm. It is recommended to maximize the number of students of the same master track in each group to ensure a similar availability between group members. You are free to use any programming language among: python, C, C++, java, javascript, ... ; but note that your code must be easily run on a standard machine and that your experiments should make fair comparisons. **The submitted code must clearly understandable to you and other persons. You must be able to explain and justify each part of the code: your understanding and ability to explain your solution will be evaluated.**

3 What to send

Each group must upload before **Thursday November 05, 22:00** a text containing the members of the group on moodle, in the `Group_Description 26-27` resource. This text **must also contain a provisional planning** giving the milestones of the project with the tasks to achieve, the repartition between the group members and the time deserved to each task, if possible the group can mention the procedures used for testing and evaluating the programs. The ability to respect the schedule will not be used for evaluating the grade, but during the defense the students must present the real planning and discuss the reasons why the project has run late. The group must also indicate the workload between the members of the group in percentage[1]. Finally, the group must include the checklist provided at the end of the document in the report and states for each question what has been done (or not). The questions of the checklist cannot be modified. The quality of the presentation (report, oral, ...) and the quality of the source code will be taken into account.

Diego Pinto-Suárez will supervise the advancement of the project, you will have 6 meetings for discussing your achievements. **Note that all the questions and mails related to the project**

[1]For a group of 4 students, if the workload was the same for every member, the group will indicate 25% for each, otherwise the group must give the distribution of the workload between the group members such that the quantities given sum up to 100%.

must be sent to him: `diego.pinto.suarez@univ-st-etienne.fr`.

The final version of the projects must be uploaded on Moodle, in the `Project_archive 26-27` resource, before **Tuesday December 15, 22:00**, in a `.zip` or `.tar.gz` archive, **well-organized**. It must include the source code of the programs, information for installing and using them, and a report in PDF format. A project defense will be scheduled on **Thursday December 17** morning or afternoon (the exact schedule will be given later), be careful to come to the defense with a working implementation and a presentation. The slides must be uploaded on moodle just before your defense on **December 17**. For your defense, you will have to prepare a short presentation of 5 minutes illustrating the main points of your work. The presentation will be followed by a series of questions.

Grade scaling:

- **At least 6/20:** The greedy and dynamic programming algorithms must be correctly implemented and rigorously evaluated on artificially generated instances. The report and source code must be clearly organized.
- **At least 10/20:** Each group member must implement a different algorithm, including brute force, branch and bound, greedy, and dynamic programming. The algorithms must be rigorously evaluated on artificially generated instances and at least four benchmark instances. The report and source code must be clearly organized, and students must be able to answer questions about their work.
- **At least 12/20:** In addition to the previous requirements, the algorithms must be evaluated on a substantial part of the benchmark collection, and at least one additional method must be implemented, such that the total number of methods exceeds the number of group members. The experimental setup must be clearly described, and the results must be analyzed and discussed.
- **At least 14/20:** In addition to the previous requirements, two additional methods must be implemented, and a larger part of the benchmark collection must be evaluated. The source code and report must be exceptionally clear and well organized. The experimental setup must be well justified, and the results must be thoroughly analyzed and discussed.
- **At least 16/20:** All the algorithms listed in Section 2 must be implemented, and a substantial part of the benchmark collection must be evaluated. The analysis must identify the types of instances on which each method performs well. Students must also propose a real-world problem that can be modeled as a Set Cover instance and solve it using their algorithms. A graphical user interface may also be proposed.

Note that for a given grade, the absence of one element can be compensated by the addition of an element associated to a higher grade. Note that if the full check list is not included in the report and commented, nor the provisional and real plannings, the grade can be divided by 2.

Be careful, the grading scale is given for information purposes. In particular it does not take into account the quality of the oral defense and the answers to questions.

Please, feel free to reread all these instructions regularly.

Checklist

1. Did you proofread your report?
2. Did you present the global objective of your work?
3. Did you present the principles of all the methods/algorithms used in your project?
4. Did you cite correctly the references to the methods/algorithms that are not from your own?
5. Did you include all the details of your experimental setup to reproduce the experimental results, and explain the choice of the parameters taken?
6. Did you provide curves, numerical results and error bars when results are run multiple times?
7. Did you comment and interpret the different results presented?
8. Did you include all the data, code, installation and running instructions needed to reproduce the results?
9. Did you engineer the code of all the programs in a unified way to facilitate the addition of new methods/techniques and debugging?
10. Did you make sure that the results different experiments and programs are comparable?
11. Did you comment sufficiently your code?
12. Did you add a thorough documentation on the code provided?
13. Did you provide the additional planning and the final planning in the report and discuss organization aspects in your work?
14. Did you describe precisely in the report the algorithms that were not seen in class?
15. Did you provide the workload percentage between the members of the group in the report?
16. Did you send the work in time?

Suggested Guideline for Grading Scale

Use this checklist to verify your progress toward different grade levels. Mark each item as completed or not completed.

Target Grade	Requirement	Yes/No
Minimum Requirements (6/20)		
6/20	Greedy Set Cover algorithm implemented and working correctly	
6/20	Dynamic programming algorithm implemented and working correctly	
6/20	Rigorous evaluation on artificially generated instances	
6/20	Report presented neatly	
6/20	Source code presented neatly	
Good Level (10/20)		
10/20	Each group member implements a different approach	
10/20	Brute-force algorithm implemented	
10/20	Branch-and-bound algorithm implemented	
10/20	Greedy algorithm implemented	
10/20	Dynamic programming algorithm implemented	
10/20	Rigorous experimental evaluation on artificially generated instances	
10/20	At least 4 benchmark instances tested	
10/20	Source code and report presented neatly	
10/20	All checklist questions answered correctly	
Very Good Level (12/20)		
12/20	All previous requirements (6/20 and 10/20) satisfied	
12/20	Algorithms evaluated on a substantial part of the benchmark collection	
12/20	Experimental setup clearly described and results analyzed	
12/20	One additional method implemented (total number of methods greater than the group size)	
Excellent Level (14/20)		
14/20	All previous requirements (up to 12/20) satisfied	
14/20	Two additional methods implemented	
14/20	A larger part of the benchmark collection evaluated	
14/20	Experimental setup clearly justified, with thorough analysis and discussion of the results	
14/20	Source code exceptionally clear and well organized	
14/20	Report exceptionally clear and well organized	
Outstanding Level (16/20)		
16/20	All algorithms listed in Section 2 implemented	
16/20	A substantial part of the benchmark collection evaluated	
16/20	Analysis identifies the types of instances on which each method performs well	
16/20	A real-world problem modeled as a Set Cover instance and solved using the implemented algorithms	
16/20	(Optional) A graphical user interface proposed	

Table 2: Suggested grading criteria for the Minimum Set Cover project.

Project Completion Checklist

This checklist must be completed and included in your report. Mark each item and provide relevant comments or observations.

✓ or ✗	Requirement	Comments/Observations
	Did you proofread your report?	
	Did you present the global objective of your work?	
	Did you present the principles of all the methods/algorithms used in your project?	
	Did you cite correctly the references to the methods/algorithms that are not from your own?	
	Did you include all the details of your experimental setup to reproduce the experimental results, and explain the choice of the parameters taken?	
	Did you provide curves, numerical results and error bars when results are run multiple times?	
	Did you comment and interpret the different results presented?	
	Did you include all the data, code, installation and running instructions needed to reproduce the results?	
	Did you engineer the code of all the programs in a unified way to facilitate the addition of new methods/techniques and debugging?	
	Did you make sure that the results of different experiments and programs are comparable?	
	Did you comment sufficiently your code?	
	Did you add a thorough documentation on the code provided?	
	Did you provide the provisional planning and the final planning in the report and discuss organization aspects in your work?	
	Did you describe precisely in the report the algorithms that were not seen in class?	
	Did you provide the workload percentage between the members of the group in the report?	
	Did you send the work in time?	

Warning: If the full checklist is not included in the report and commented on, nor the provisional (the one you are supposed to sent by Thursday November 05) and real plannings (the real task distribution), the grade can be divided by 2.